

1000000，改善了程序的可读性，也方便修改（假设题目改成求末 5 位正整数之积）。

这个程序真正的特别之处在于计时函数 `clock()` 的使用。该函数返回程序目前为止运行的时间。这样，在程序结束之前调用此函数，便可获得整个程序的运行时间。这个时间除以常数 `CLOCKS_PER_SEC` 之后得到的值以“秒”为单位。

提示 2-17：可以使用 `time.h` 和 `clock()` 函数获得程序运行时间。常数 `CLOCKS_PER_SEC` 和操作系统相关，请不要直接使用 `clock()` 的返回值，而应总是除以 `CLOCKS_PER_SEC`。

输入“20”，按 Enter 键后，系统瞬间输出了答案 820313。但是，输出的 Time used 居然不是 0！其原因在于，键盘输入的时间也被计算在内——这的确是程序启动之后才进行的。为了避免输入数据的时间影响测试结果，可使用一种称为“管道”的小技巧：在 Windows 命令行下执行 `echo 20 | abc`，操作系统会自动把 20 输入，其中 `abc` 是程序名^①。如果不知道如何操作命令行，请参考附录 A。笔者建议每个读者都熟悉命令行操作，包括 Windows 和 Linux。

在尝试了多个 n 之后，得到了一张表，如表 2-1 所示。

表 2-1 程序 2-8 的输出结果与运行时间表

n	20	40	80	160	1600	6400	12800	25600	51200
答案	820313	940313	940313	940313	940313	940313	940313	940313	940313
时间	<0.01	<0.01	<0.01	<0.01	0.05	0.70	2.70	11.08	43.72

由表 2-1 可知：第一，程序的运行时间大致和 n 的平方成正比（因为 n 每扩大 1 倍，运行时间近似扩大 4 倍）。甚至可以估计 $n=10^6$ 时，程序大致需要近 5 个小时才能执行完。

提示 2-18：很多程序的运行时间与规模 n 存在着近似的简单关系。可以通过计时函数来发现或验证这一关系。

第二，从 40 开始，答案始终不变。这是真理还是巧合？聪明的读者也许已经知道了：25！末尾有 6 个 0，所以从第 5 项开始，后面的所有项都不会影响和的末 6 位数字——只需要在程序的最前面加一条语句“`if(n>25) n = 25;`”，效率和溢出都将不存在问题。

本节展示了循环结构程序设计中最常见的两个问题：算术运算溢出和程序效率低下。这两个问题都不是那么容易解决的，将在后面章节中继续讨论。另外，本节中介绍的两个工具——输出中间结果和计时函数，都是相当实用的。

2.4 算法竞赛中的输入输出框架

例题 2-5 数据统计

输入一些整数，求出它们的最小值、最大值和平均值（保留 3 位小数）。输入保证这些数都是不超过 1000 的整数。

^① Linux 下需要输入“`echo | ./abc`”，因为在默认情况下，当前目录不在可执行文件的搜索路径中。

样例输入：

2 8 3 5 1 7 3 6

样例输出：

1 8 4.375

【分析】

如果是先输入整数 n ，然后输入 n 个整数，相信读者能够写出程序。关键在于：整数的个数是不确定的。下面直接给出程序：

程序 2-9 数据统计（有 bug）

```
#include<stdio.h>
int main()
{
    int x, n = 0, min, max, s = 0;
    while(scanf("%d", &x) == 1)
    {
        s += x;
        if(x < min) min = x;
        if(x > max) max = x;
        n++;
    }
    printf("%d %d %.3f\n", min, max, (double)s/n);
    return 0;
}
```

看看这个程序多了些什么内容？scanf 函数有返回值？对，它返回的是成功输入的变量个数，当输入结束时，scanf 函数无法再次读取 x ，将返回 0。

下面进行测试。输入“2 8 3 5 1 7 3 6”，按 Enter 键，但未显示结果。难道程序速度太慢？其实程序正在等待输入。还记得 scanf 的输入格式吗？空格、TAB 和回车符都是无关紧要的，所以按 Enter 键并不意味着输入的结束。那如何才能告诉程序输入结束了呢？

提示 2-19：在 Windows 下，输入完毕后先按 Enter 键，再按 Ctrl+Z 键，最后再按 Enter 键，即可结束输入。在 Linux 下，输入完毕后按 Ctrl+D 键即可结束输入。

输入终于结束了，但输出却是“1 2293624 4.375”。这个 2293624 是从何而来？当用 -O2 编译（读者可阅读附录 A 了解 -O2）后答案变成了 1 10 4.375，和刚才不一样！换句话说，这个程序的运行结果是不确定的。在读者自己的机器上，答案甚至可能和上述两个都不同。

根据“输出中间结果”的方法，读者不难验证下面的结论：变量 max 在一开始就等于 2293624（或者 10），自然无法更新为比它小的 8。

提示 2-20：变量在未赋值之前的值是不确定的。特别地，它不一定等于 0。

解决的方法就很清楚了：在使用之前赋初值。由于 `min` 保存的是最小值，其初值应该是一个很大的数；反过来，`max` 的初值应该是一个很小的数。一种方法是定义一个很大的常数，如 `INF = 1000000000`，然后让 `max = -INF`，而 `min = INF`，另一种方法是先读取第一个整数 `x`，然后令 `max = min = x`。这样的好处是避免了人为的“假想无穷大”值，程序更加优美；而 `INF` 这样的常数有时还会引起其他问题，如“无限大不够大”，或者“运算溢出”，后面还会继续讨论这个问题。

上面的程序并不是很方便：每次测试都要手动输入许多数。尽管可以用前面讲的管道的方法，但数据只是保存在命令行中，仍然不够方便。

一个好的方法是用文件——把输入数据保存在文件中，输出数据也保存在文件中。这样，只要事先把输入数据保存在文件中，就不必每次重新输入了；数据输出在文件中也避免了“输出太多，一卷屏前面的就看不见了”这样的尴尬，运行结束后，慢慢浏览输出文件即可。如果有标准答案文件，还可以进行文件比较^①，而无须编程人员逐个检查输出是否正确。事实上，几乎所有算法竞赛的输入数据和标准答案都是保存在文件中的。

使用文件最简单的方法是使用输入输出重定向，只需在 `main` 函数的入口处加入以下两条语句：

```
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
```

上述语句将使得 `scanf` 从文件 `input.txt` 读入，`printf` 写入文件 `output.txt`。事实上，不只是 `scanf` 和 `printf`，所有读键盘输入、写屏幕输出的函数都将改用文件。尽管这样做很方便，并不是所有算法竞赛都允许用程序读写文件。甚至有的竞赛允许访问文件，但不允许用 `freopen` 这样的重定向方式读写文件。参赛之前请仔细阅读文件读写的相关规定。

提示 2-21：请在比赛之前了解文件读写的相关规定：是标准输入输出（也称标准 I/O，即直接读键盘、写屏幕），还是文件输入输出？如果是文件输入输出，是否禁止用重定向方式访问文件？

多年来，无数选手因文件相关问题丢掉了大量分数。一个普适的原则是：详细阅读比赛规定，并严格遵守。例如，输入输出文件名和程序名往往都有着严格规定，不要弄错大小写，不要拼错文件名，不要使用绝对路径或相对路径。

例如，如果题目规定程序名称为 `test`，输入文件名为 `test.in`，输出文件名为 `test.out`，就不要犯以下错误。

错误 1：程序存为 `t1.c`（应该改成 `test.c`）。

错误 2：从 `input.txt` 读取数据（应该从 `test.in` 读取）。

错误 3：从 `tset.in` 读取数据（拼写错误，应该从 `test.in` 读取）。

错误 4：数据写到 `test.ans`（扩展名错误，应该是 `test.out`）。

错误 5：数据写到 `c:\\contest\\test.out`（不能加路径，哪怕是相对路径。文件名应该只有

^① 在 Windows 中可以使用 `fc` 命令，而在 Linux 中可以使用 `diff` 命令。

8 个字符：test.out）。

提示 2-22：在算法竞赛中，选手应严格遵守比赛的文件名规定，包括程序文件名和输入输出文件名。不要弄错大小写，不要拼错文件名，不要使用绝对路径或相对路径。

当然，这些错误都不是选手故意犯下的。前面说过，利用文件是一种很好的自我测试方法，但如果比赛要求采用标准输入输出，就必须在自我测试完毕之后删除重定向语句。选手比赛时一紧张，就容易忘记将其删除。

有一种方法可以在本机测试时用文件重定向，但一旦提交到比赛，就自动“删除”重定向语句。代码如下：

程序 2-10 数据统计（重定向版）

```
#define LOCAL
#include<stdio.h>
#define INF 1000000000
int main()
{
#ifdef LOCAL
    freopen("data.in", "r", stdin);
    freopen("data.out", "w", stdout);
#endif
    int x, n = 0, min = INF, max = -INF, s = 0;
    while(scanf("%d", &x) == 1)
    {
        s += x;
        if(x < min) min = x;
        if(x > max) max = x;
        /*
        printf("x = %d, min = %d, max = %d\n", x, min, max);
        */
        n++;
    }
    printf("%d %d %.3f\n", min, max, (double)s/n);
    return 0;
}
```

这是一份典型的比赛代码，包含了几个特殊之处：

- ❑ 重定向的部分被写在了`#ifdef`和`#endif`中。其含义是：只有定义了符号 `LOCAL`，才编译两条 `freopen` 语句。
- ❑ 输出中间结果的 `printf` 语句写在了注释中——它在最后版本的程序中不应该出现，但是又舍不得删除它（万一发现了新的 `bug`，需要再次用它输出中间信息）。将其注释的好处是：一旦需要时，把注释符去掉即可。

上面的代码在程序首部就定义了符号 `LOCAL`，因此在本机测试时使用重定向方式读写文件。如果比赛要求读写标准输入输出，只需在提交之前删除 `#define LOCAL` 即可。一个更好的方法是在编译选项而不是程序里定义这个 `LOCAL` 符号（不知道如何在编译选项里定义符号的读者请参考附录 A），这样，提交之前不需要修改程序，进一步降低了出错的可能。

提示 2-23: 在算法竞赛中，有经验的选手往往会使用条件编译指令并且将重要的测试语句注释掉而非删除。

如果比赛要求用文件输入输出，但禁止用重定向的方式，又当如何呢？程序如下：

程序 2-11 数据统计（fopen 版）

```
#include<stdio.h>
#define INF 1000000000
int main()
{
    FILE *fin, *fout;
    fin = fopen("data.in", "rb");
    fout = fopen("data.out", "wb");
    int x, n = 0, min = INF, max = -INF, s = 0;
    while(fscanf(fin, "%d", &x) == 1)
    {
        s += x;
        if(x < min) min = x;
        if(x > max) max = x;
        n++;
    }
    fprintf(fout, "%d %d %.3f\n", min, max, (double)s/n);
    fclose(fin);
    fclose(fout);
    return 0;
}
```

虽然新内容不少，但也很直观：先声明变量 `fin` 和 `fout`（暂且不用考虑 `FILE *`），把 `scanf` 改成 `fscanf`，第一个参数为 `fin`；把 `printf` 改成 `fprintf`，第一个参数为 `fout`，最后执行 `fclose`，关闭两个文件。

提示 2-24: 在算法竞赛中，如果不允许使用重定向方式读写数据，应使用 `fopen` 和 `fscanf/fprintf` 进行输入输出。

重定向和 `fopen` 两种方法各有优劣。重定向的方法写起来简单、自然，但是不能同时读写文件和标准输入输出；`fopen` 的写法稍显繁琐，但是灵活性比较大（例如，可以反复打开并读写文件）。顺便说一句，如果想把 `fopen` 版的程序改成读写标准输入输出，只需赋值“`fin =`

stdin; fout = stdout;”即可，不要调用 fopen 和 fclose^①。

对文件输入输出的讨论到此结束，本书剩余部分的所有题目均使用标准输入输出。

例题 2-6 数据统计 II

输入一些整数，求出它们的最小值、最大值和平均值（保留 3 位小数）。输入保证这些数都是不超过 1000 的整数。

输入包含多组数据，每组数据第一行是整数个数 n ，第二行是 n 个整数。 $n=0$ 为输入结束标记，程序应当忽略这组数据。相邻两组数据之间应输出一个空行。

样例输入：

```
8
2 8 3 5 1 7 3 6
4
-4 6 10 0
0
```

样例输出：

Case 1: 1 8 4.375

Case 2: -4 10 3.000

【分析】

本题和例题 2-5 本质相同，但是输入输出方式有了一定的变化。由于这样的格式在算法竞赛中非常常见，这里直接给出代码：

程序 2-12 数据统计 II（有 bug）

```
#include<stdio.h>
#define INF 1000000000
int main()
{
    int x, n = 0, min = INF, max = -INF, s = 0, kase = 0;
    while(scanf("%d", &n) == 1 && n)
    {
        int s = 0;
        for(int i = 0; i < n; i++) {
            scanf("%d", &x);
            s += x;
            if(x < min) min = x;
            if(x > max) max = x;
        }
        if(kase) printf("\n");
    }
}
```

^① 有读者可能试过用 fopen("con", "r")的方法打开标准输入输出，但这个方法并不是可移植的——它在 Linux 下是无效的。

```

printf("Case %d: %d %d %.3f\n", ++kase, min, max, (double)s/n);
}
return 0;
}

```

聪明的读者，你能看懂其中的逻辑吗？上面的程序有几个要点。首先是输入循环。题目说了 $n=0$ 为输入标记，为什么还要判断 `scanf` 的返回值呢？答案是为了鲁棒性(robustness)。算法竞赛中题目的输入输出是人设计的，难免会出错。有时会出现题目指明以 $n=0$ 为结束标记而真实数据忘记以 $n=0$ 结尾的情形。虽然比赛中途往往会修改这一错误，但在 ACM/ICPC 等时间紧迫的比赛中，如果程序能自动处理好有瑕疵的数据，会节约大量不必要的时间浪费。

提示 2-25：在算法竞赛中，偶尔会出现输入输出错误的情况。如果程序鲁棒性强，有时能在数据有瑕疵的情况下仍然给出正确的结果。程序的鲁棒性在工程中也非常重要。

下一个要点是 `kase` 变量的使用。不难看出它是“当前数据编号”计数器。当输出第 2 组或以后的结果时，会在前面加一个空行，符合题目“相邻两组数据的输出以空行隔开”的规定。注意，最后一组数据的输出会以回车符结束，但之后不会有空行。不同的题目会有不同的规定，请读者仔细阅读题目。

像本题这样“多组数据”的题目数不胜数。例如，ACM/ICPC 总决赛就只有一个输入文件，包含多组数据。即使是 NOI/IOI 这样多输入文件的比赛，有时也会出现一个文件多组数据的情况。例如，有的题目输出只有 Yes 和 No 两种，如果一个文件里只有一组数据，又是每个文件分别给分，一个随机输出 Yes/No 的程序平均情况下能得 50 分，而一个把 Yes 打成 yes, No 打成 no 的程序却只有 0 分^①。

接下来是找 bug 时间。上面的程序对于样例输入输出可以得到正确的结果，但它真的是正确的吗？在样例输入的最后增加第 3 组数据：1 0，会看到这样的输出：

Case 3: -4 10 0.000

相信读者已经意识到问题出在哪里了：`min` 和 `max` 没有“重置”，仍然是上个数据结束后的值。

提示 2-26：在多数数据的题目中，一个常见的错误是：在计算完一组数据后某些变量没有重置，影响到下组数据的求解。

解决方法很简单，把 `min` 和 `max` 定义在 `while` 循环中即可，这样每次执行循环体时，会新声明和初始化 `min` 和 `max`。细心的读者也许注意到了另外一个问题：为什么第 3 个数（累加和）是对的呢？原因在于：循环体内部也定义了一个 `s`，把 `main` 函数里定义的 `s` 给“屏蔽”了。

提示 2-27：当嵌套的两个代码块中有同名变量时，内层的变量会屏蔽外层变量，有时会引起十分隐蔽的错误。

^① 也不总是如此。有些比赛会善意地把这种只是格式不对的结果判成“正确”。可惜这样的比赛非常少。